

U.S. SEMICONDUCTOR: TRADE & LABOR ANALYSIS

Company Census File — Cleaning Process & Validation (20260121) | 2025 Q3

Overview

This document logs the cleaning process applied to the Company Census File (20260121), an administrative dataset containing carrier identity, fleet metrics, safety and compliance information, and cargo specialization flags. The raw file contained over 2 million rows and more than 150 columns, many of which were administrative, redundant, or too granular for analysis.

The goal of this cleaning step was to reduce the dataset to a lean, consistent structure that preserves the analytical core — carrier identity, fleet size, safety record, and cargo type — while minimizing memory usage and ensuring full reproducibility and audit readiness.

Source & Dataset Information

Attribute	Value
Data provider	Federal Motor Carrier Safety Administration (FMCSA) via BLS-linked datasets
Publication	Company Census File — 20260121
Format	CSV (.csv) — raw administrative file
Records	2,000,000+ rows, 150+ columns in raw form
Output	Cleaned_Company_Census_File_20260121.csv

Cleaning Steps Summary

The following eight steps were applied sequentially to transform the raw Company Census file into an analysis-ready dataset. Each step is documented with the corresponding code and the rationale for the decision.

Step	Action	Code	Rationale
1	Read raw dataset	<code>pd.read_csv(..., low_memory=False)</code>	Loads the full CSV with dtype scanning enabled to avoid mixed-type warnings on large files
2	Drop redundant columns	<code>df.drop(columns=cols_to_drop)</code>	Removes administrative metadata, contact details, ownership transfer fields, and duplicate mailing addresses

3	Handle missing values	<code>df.fillna('NaN') / replace with np.nan</code>	Standardizes missing values during cleaning, then converts placeholders to proper NaN for numeric operations
4	Downcast numeric types	<code>pd.to_numeric(downcast='integer'/'float')</code>	Converts int64 to int32 and float64 to float32, reducing memory footprint by 30-50%
5	Optimize cargo flags	<code>df[cargo_cols].astype('category')</code>	Converts all CRGO_* binary flag columns to category dtype for faster filtering and grouping
6	Validate uniqueness	<code>df[['DOT_NUMBER']].duplicated().sum()</code>	Confirms each carrier DOT number is unique to ensure dataset integrity
7	Profile dataset	<code>df.describe() / df.dtypes / df.info()</code>	Generates descriptive statistics and memory usage report to confirm cleaning outcomes
8	Save cleaned dataset	<code>df.to_csv('Cleaned_...csv', index=False)</code>	Exports the final cleaned file without the pandas index for clean downstream ingestion

Step-by-Step Detail

Step 1 — Reading the Raw Dataset

```
df = pd.read_csv('Company_Census_File_20260121.csv', low_memory=False)
```

The raw file was loaded using `low_memory=False` to ensure pandas scans entire columns before assigning data types. This prevents `DtypeWarning` errors from mixed-type columns, which are common in large administrative files with sparse or inconsistently populated fields.

Step 2 — Dropping Redundant Columns

```
df = df.drop(columns=cols_to_drop)
```

Over 50 non-analytical columns were removed across four categories:

- Administrative metadata: MCS150_DATE, ADD_DATE, REVIEW_ID, and similar tracking fields not needed for analysis.
- Contact details: PHONE, FAX, EMAIL_ADDRESS, and related fields with no role in labor or fleet analysis.
- Hyper-granular ownership and transfer fields: OWNTRUCK, TRMTRUCK, TRPTRUCK, and equivalents that overlap with higher-level fleet metrics.
- Duplicate mailing address fields: CARRIER_MAILING_* columns that duplicate the primary address fields.

This reduction brought the dataset down to core analytical fields covering carrier identity, fleet size, safety compliance, and cargo specialization.

Step 3 — Handling Missing Values

```
df = df.fillna('NaN')
df.replace('NaN', np.nan, inplace=True)
```

Missing values were first standardized to the string 'NaN' during the initial cleaning pass to ensure consistent handling across mixed-type columns. They were then replaced with proper numpy NaN values to ensure compatibility with all pandas and numpy operations in downstream analysis — including aggregations, numeric conversions, and statistical profiling.

Step 4 — Downcasting Numeric Columns

```
df_int = df.select_dtypes(include=['int64'])
df[df_int.columns] = df_int.apply(pd.to_numeric, downcast='integer')
df_float = df.select_dtypes(include=['float64'])
df[df_float.columns] = df_float.apply(pd.to_numeric, downcast='float')
```

All integer columns were downcast from int64 to int32, and all float columns from float64 to float32. Fleet counts, driver totals, and mileage values are well within the range of 32-bit numeric types, making this a lossless operation. The result is a 30–50% reduction in memory footprint, improving performance when working with the full 2-million-row dataset.

Step 5 — Optimizing Cargo Flag Columns

```
cargo_cols = [col for col in df.columns if col.startswith('CRGO_')]
df[cargo_cols] = df[cargo_cols].astype('category')
```

All cargo specialization columns — including CRGO_GENFREIGHT, CRGO_MEAT, CRGO_CARGOOTH, and others — were converted to the pandas category dtype. These columns contain binary flag values (Y/N or 0/1) across a limited set of distinct values, making them ideal candidates for categorical encoding. The conversion reduces memory usage and accelerates filtering and grouping operations used in cargo-type segmentation.

Step 6 — Validating Uniqueness

```
print(df[['DOT_NUMBER']].duplicated().sum())
```

A duplicate check on the DOT_NUMBER field confirmed that each carrier identifier is unique in the cleaned dataset. This validation ensures downstream joins and aggregations by carrier are reliable and that no carrier record is double-counted in fleet or compliance summaries.

Step 7 — Profiling the Dataset

```
print(df.describe(include='all'))
print(df.dtypes)
print(df.info(memory_usage='deep'))
```

Descriptive statistics, column data types, and a deep memory usage report were generated to confirm data integrity and the effectiveness of all prior cleaning steps. This profiling step serves as the final audit checkpoint before the cleaned file is exported.

Step 8 — Saving the Cleaned Dataset

```
df.to_csv('Cleaned_Company_Census_File_20260121.csv', index=False)
```

The cleaned dataset was exported to CSV format without the pandas index, producing a clean file ready for downstream analysis tools. The output preserves all retained analytical fields in their optimized data types.

Key Decisions and Rationale

- Dropped 50+ non-analytical columns to focus the dataset on carrier identity, fleet size, safety compliance, and cargo specialization.
- Downcasted numeric types to reduce memory usage by approximately 30–50%, enabling efficient analysis at scale.
- Converted cargo flag columns to category dtype for faster filtering and grouping across cargo-type segmentation queries.
- Standardized missing values as NaN to ensure full compatibility with pandas and numpy statistical operations.
- Validated uniqueness of DOT_NUMBER to confirm dataset integrity and reliable downstream joins by carrier.
- Documented all steps with code for full reproducibility and audit readiness.

Data Source

[Quarterly Census of Employment and Wages — U.S. Bureau of Labor Statistics](#)